

PROJECT REPORT OF INDUSTRY ORIENTED HANDS ON EXPERIENCE (IOHE)
ON
UB E-Health: AI-Powered Integrated Healthcare and Hospital Management
System
submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING
In
COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Name	Roll no.
Shaishav	2211985046
Sangram Jyoti Patra	2211985045



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY

CONTENTS

S.No.	Title	Page No.
	Acknowledgement	4
	Abstract	5
1	Introduction	6
2	Methodology	9
3	Tools and Technologies	11
4	Diagrams	15
5	Output/Results	17
6	Conclusion and Future Scope	21
	References	24

DECLARATION

I hereby certify that the work which is being presented in the project report entitled
“**UB E-Health: AI Powered Integrated Healthcare and Hospital Management System**” in
partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer
Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara
University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic
record of my own work carried out under the supervision of **Dr. Rajat Takkar**. The matter presented in
this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Shaishav, Sangram Jyoti Patra

Date : 29/04/2024

(University Roll No.: 2211985046, 2211985045)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and
belief.

Dr. Rajat Takkar

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who supported and guided me throughout the development of this project. I am deeply thankful to my project guide and faculty members for their invaluable guidance, constructive feedback, and continuous encouragement during this work. Their expertise and insights helped shape the direction and quality of this project. I extend my appreciation to the open-source community whose libraries and tools -- including React.js, Node.js, MongoDB, Socket.io, and the Google Gemini API -- formed the foundation of this system. Without these freely available resources, a project of this scope would not have been achievable. I am grateful to my institution for providing the infrastructure, laboratory facilities, and academic environment that made this project possible.

ABSTRACT

UB E-Health Management Hub is a comprehensive, full-stack web application designed to digitize and streamline hospital management operations. Built on the MERN stack (MongoDB, Express.js, React.js, Node.js), the system provides role-specific dashboards and workflows for four user types: patients, doctors, laboratory personnel, and administrators.

The platform addresses the critical gap in conventional healthcare systems that rely on manual processes, paper-based records, and fragmented communication. Key innovations include a dual-mode AI report interpretation engine powered by Google Gemini with an offline fallback to locally hosted Ollama models, a real-time doctor-patient chat system built on Socket.io and Redis, and an integrated laboratory management module with home sample collection.

The system implements JWT-based role authentication, Razorpay payment integration, automated email notifications via cron jobs, digital prescription management with PDF generation, health trend analytics, and a document management system. The AI interpretation feature uniquely combines cloud AI (Gemini), local AI (Ollama), and a rule-based fallback engine, ensuring availability in both online and offline environments.

The result is a production-ready prototype that reduces appointment scheduling time, improves patient understanding of medical reports, enables asynchronous doctor-patient communication, and centralizes all healthcare records in a single secure platform.

Keywords: MERN Stack, Healthcare Management, AI Report Interpretation, Real-Time Chat, Socket.io, Google Gemini, Ollama, JWT Authentication, Razorpay, Laboratory Management, Telemedicine

INTRODUCTION

1.1 Problem Background

The conventional healthcare system continues to rely heavily on manual processes, paper-based records, and fragmented communication channels between patients, doctors, and laboratory personnel.

Patients face significant difficulties in scheduling appointments, accessing their medical reports, and understanding complex clinical terminology without professional assistance. A patient receiving a lab report with values like HbA1c, eGFR, or TSH has no immediate way to understand what these numbers mean for their health without waiting for a doctor consultation.

Doctors struggle with disorganized patient histories, inefficient prescription workflows, and the inability to respond to minor patient queries without scheduling a full appointment. Laboratory staff manage test requests manually, leading to delays, lost samples, and errors in result entry.

The absence of a unified digital platform results in:

- Poor coordination between departments
- Increased administrative burden on staff
- Delayed access to medical information for patients
- Unnecessary in-person visits for minor queries
- Reduced quality of patient care, particularly in resource-limited or offline environments

1.2 Objective of the Project

The primary objective of this project is to design and develop a comprehensive, role-based digital healthcare management platform that:

1. Streamlines hospital operations through digital workflows
2. Enables AI-assisted medical report interpretation in plain patient-friendly language
3. Facilitates real-time communication between all stakeholders

4. Provides offline AI capability for resource-limited environments
5. Integrates payment processing, notifications, and document management within a single unified system
6. Ensures data security through JWT authentication, bcrypt hashing, and input sanitization

1.3 Scope of the Project

The project covers the following scope:

IN SCOPE:

- Web-based application accessible via modern browsers
- Four user roles: Patient, Doctor, Lab Personnel, Admin
- Appointment booking with online payment (Razorpay)
- Laboratory test management with home service option
- AI-powered report interpretation (Gemini + Ollama fallback)
- Real-time doctor-patient messaging (Socket.io)
- Prescription management with PDF generation
- Health trend analytics and visualization
- Document upload and sharing
- Email notification system with cron scheduling
- Ambulance service management

OUT OF SCOPE:

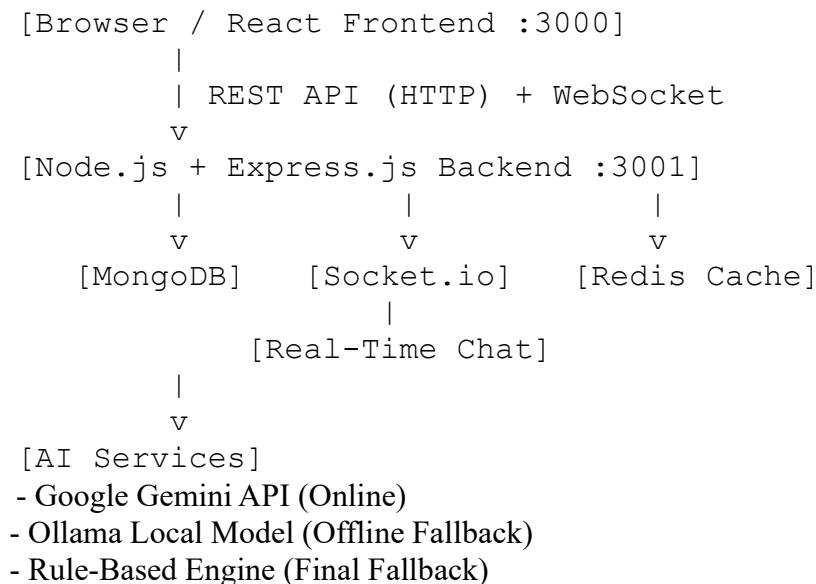
- Mobile native application (iOS/Android)
- Video consultation (WebRTC)
- Integration with external hospital ERP systems
- Medical imaging analysis (X-ray, MRI interpretation)

1.4 System Overview

UB E-Health Management Hub is a full-stack web application built on the MERN stack. The system provides role-specific dashboards and workflows for four user types.

Patients can book appointments and lab tests online, receive AI-interpreted medical reports in plain language, and communicate with their assigned doctor in real time. Doctors can manage patient records, write clinical reports, and issue digital prescriptions. Laboratory staff process test requests and enter results directly into the system. Administrators oversee all users and system operations through a centralized control panel.

System Architecture (Text Representation):



METHODOLOGY

2.1 Development Methodology

This project was developed using an Agile-inspired iterative development approach. Rather than following a strict waterfall model, features were developed in incremental sprints, with each sprint delivering a working, testable module.

The development followed this sequence:

Sprint 1: Core Authentication and User Management

- Patient, Doctor, Admin, Lab Personnel registration and login
- JWT-based authentication middleware
- Role-based route protection

Sprint 2: Appointment and Lab Test Management

- Appointment booking with slot management
- Lab test booking with home service option
- Razorpay payment integration
- Email notification system

Sprint 3: Medical Records and Prescriptions

- Doctor report creation with vitals and diagnosis
- Prescription management with PDF generation
- Document upload and sharing system

Sprint 4: AI Integration

- Google Gemini API integration for report interpretation
- Ollama local AI fallback implementation
- Rule-based fallback engine
- Health chatbot assistant

Sprint 5: Real-Time Communication

- Socket.io server setup and event handling
- Redis integration for caching and presence
- Doctor-patient chat interface
- Typing indicators and read receipts

Sprint 6: Analytics, Monitoring, and Polish

- Health trends and analytics dashboard
- Health score calculation system
- Admin dashboard and management pages
- Responsive design and UI improvements

TOOLS AND TECHNOLOGIES

3.1 Technology Stack Overview

The project is built on the MERN stack with additional technologies for real-time communication, AI integration, and payment processing.

Layers	Stack
PRESENTATION LAYER	React.js Redux CSS
APPLICATION LAYER	Node.js Express.js Socket.io
DATA LAYER	MongoDB Mongoose Redis
EXTERNAL SERVICES	Gemini AI Ollama Razorpay

3.2 Frontend Technologies

React.js (v18)

React is a JavaScript library for building user interfaces. It uses a component-based architecture where each UI element is an independent, reusable component. The virtual DOM ensures efficient updates by only re-rendering changed components. Reason for choice: Component reusability, large ecosystem, excellent performance for dynamic UIs.

Redux

Redux is a predictable state management library. It provides a centralized store for all application state, making data flow unidirectional and predictable. Actions describe what happened; reducers specify how state changes in response. Reason for choice: Needed for managing complex shared state across many components (auth, appointments, lab reports, chat).

React Router v6

Client-side routing library enabling navigation between pages without full page reloads.

Protected routes redirect unauthenticated users to the login page.

Axios

Promise-based HTTP client for making API calls from the frontend to the backend.

Supports request/response interceptors for automatic token attachment.

Chart.js with react-chartjs-2

Used for rendering interactive health trend charts (line charts for vitals over time).

Provides responsive, animated charts.

jsPDF

Client-side PDF generation library. Used for generating appointment receipts, prescription PDFs, and medical report exports directly in the browser without server involvement.

React-Toastify

Notification toast library for displaying success, error, and info messages to users in a non-intrusive way.

socket.io-client

Frontend Socket.io library for establishing WebSocket connections to the backend for real-time chat functionality.

3.3 Backend Technologies

Node.js

JavaScript runtime built on Chrome V8 engine. Enables running JavaScript on the server side. Non-blocking, event-driven architecture makes it ideal for I/O-intensive applications like real-time chat and API servers.

Express.js (v4)

Minimal and flexible Node.js web application framework. Provides routing, middleware support, and HTTP utilities. All REST API endpoints are defined using Express Router.

Socket.io (v4)

Real-time bidirectional event-based communication library. Enables the doctor-patient chat feature with WebSocket connections, automatic fallback to HTTP long-polling, and room-based message routing.

JSON Web Token (JWT)

Compact, URL-safe token format for authentication. On login, the server signs a token containing userId and role. The client sends this token with every request; the server verifies it. Stateless -- no server-side session storage required.

bcrypt

Password hashing library using the bcrypt algorithm with salt. Passwords are hashed before storage; comparison is done by hashing the input and comparing with the stored hash. Salt prevents rainbow table attacks.

Multer

Node.js middleware for handling multipart/form-data (file uploads). Used for medical document uploads and chat file attachments. Configures storage destination and filename.

Nodemailer

Node.js module for sending emails. Configured with Gmail SMTP for sending appointment confirmations, medication reminders, and lab test notifications.

node-cron

Task scheduler for Node.js. Used for running automated jobs: appointment reminders (9 AM daily), medication reminders (8 AM daily), lab test reminders (10 AM Monday), and age updates (midnight daily).

Winston

Logging library used for audit logging in the chat system. Logs all message events, file uploads, authentication failures, and security events with timestamps and severity levels.

3.4 Database Technologies

MongoDB

NoSQL document database storing data as BSON (Binary JSON). Chosen for its flexible schema, which accommodates the varying structure of medical data. Supports horizontal scaling and integrates naturally with Node.js.

Mongoose (v6)

Object Data Modeling (ODM) library for MongoDB and Node.js. Provides schema definition, validation, middleware hooks, and the `populate()` method for document references (similar to SQL JOINS).

Redis (via ioredis)

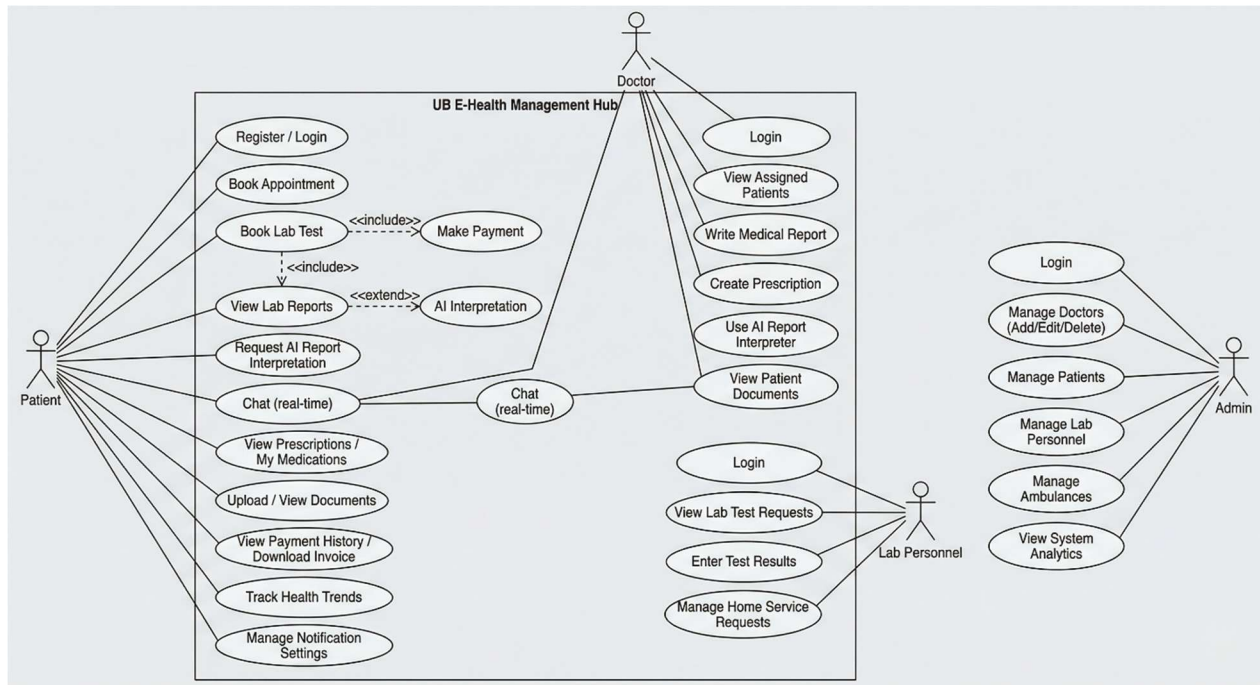
In-memory data structure store used for:

- Caching user presence (online/offline status)
- Storing typing indicators with TTL
- Rate limiting message sending
- Socket.io adapter for horizontal scaling

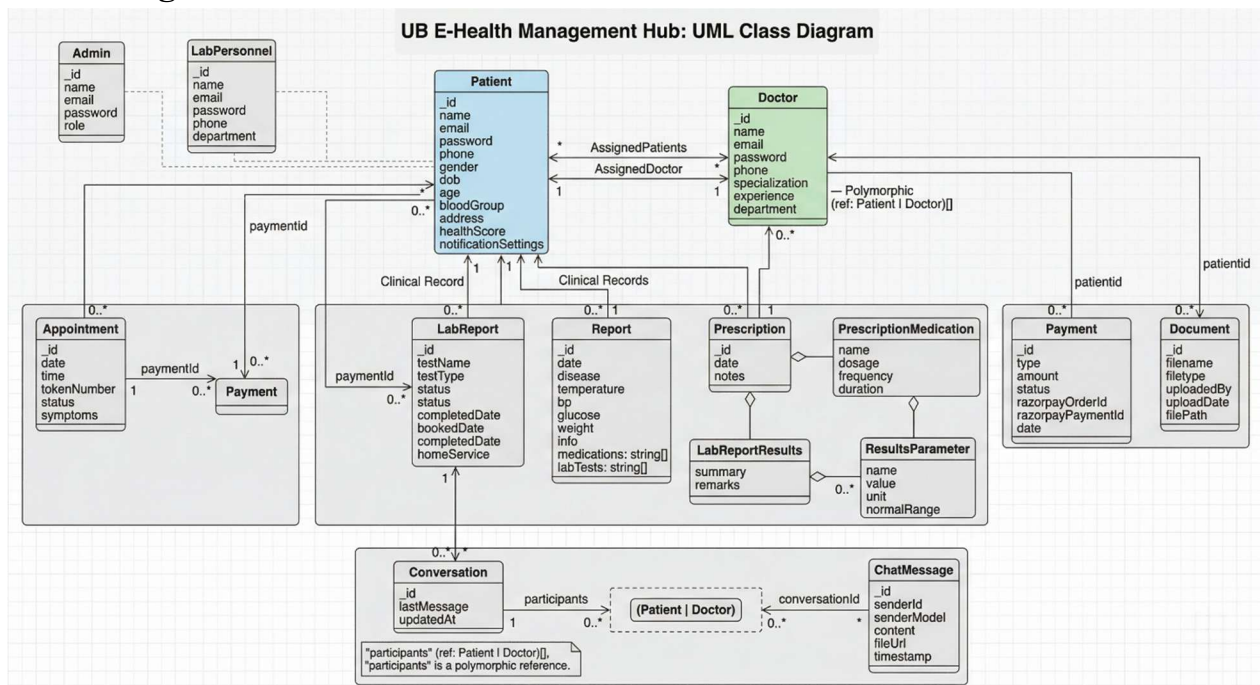
Redis is significantly faster than MongoDB for temporary, frequently-accessed data.

Diagrams

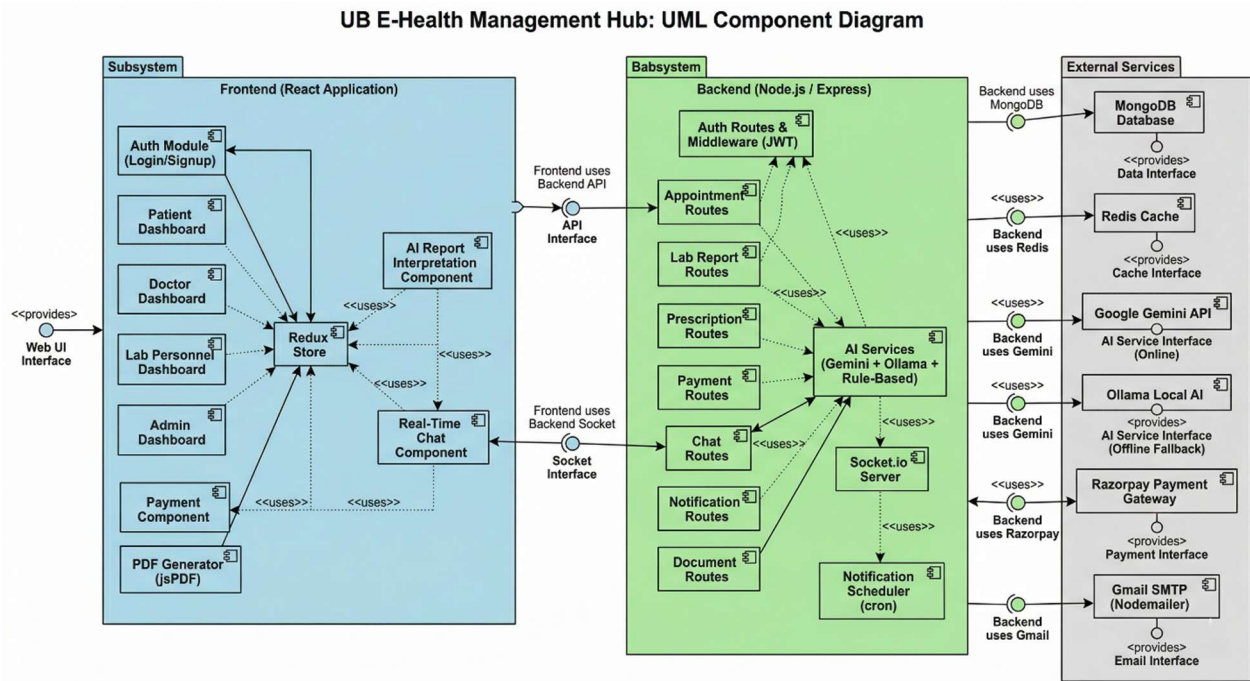
USE CASE DIAGRAM:



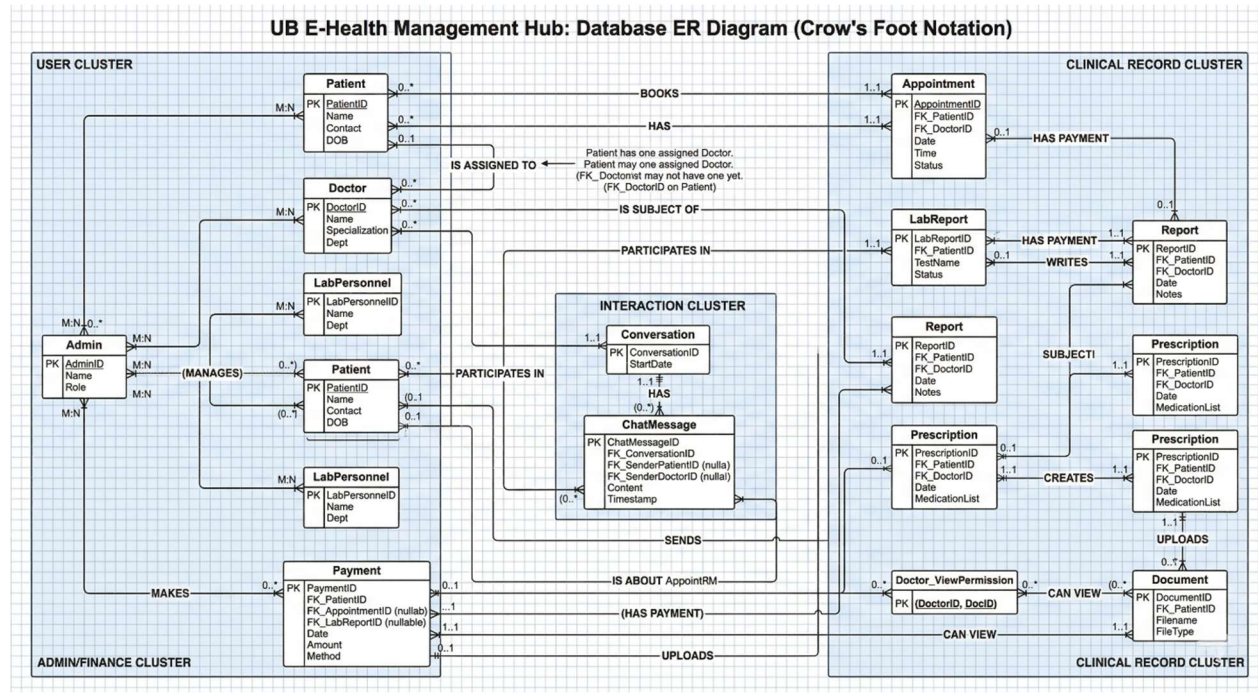
Class Diagram:



Component Diagram:

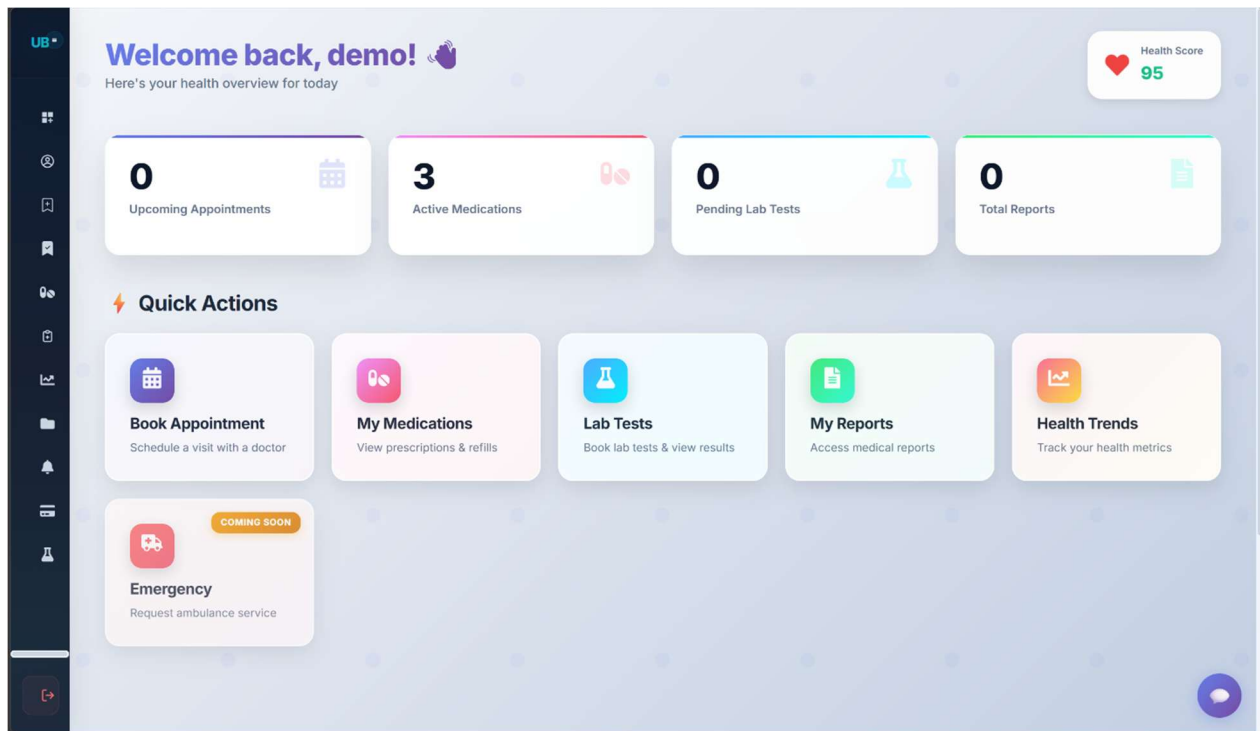


ER Diagram:



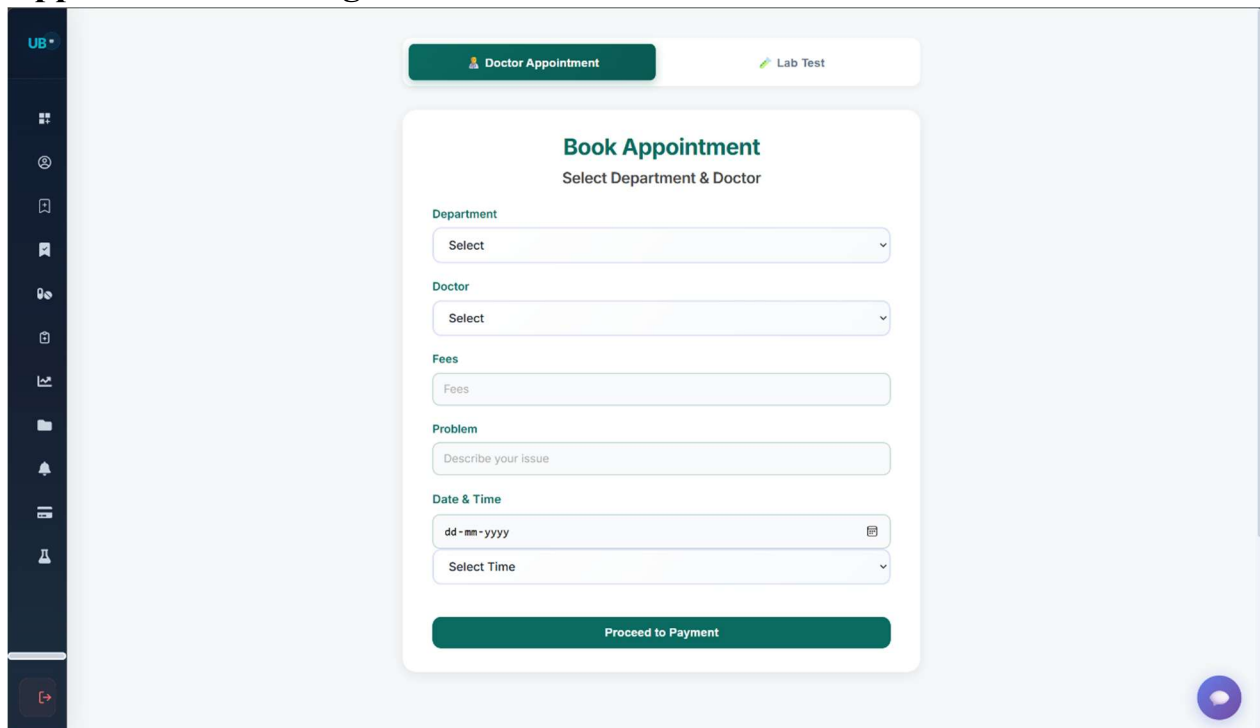
Output/Results

Patient Dashboard:



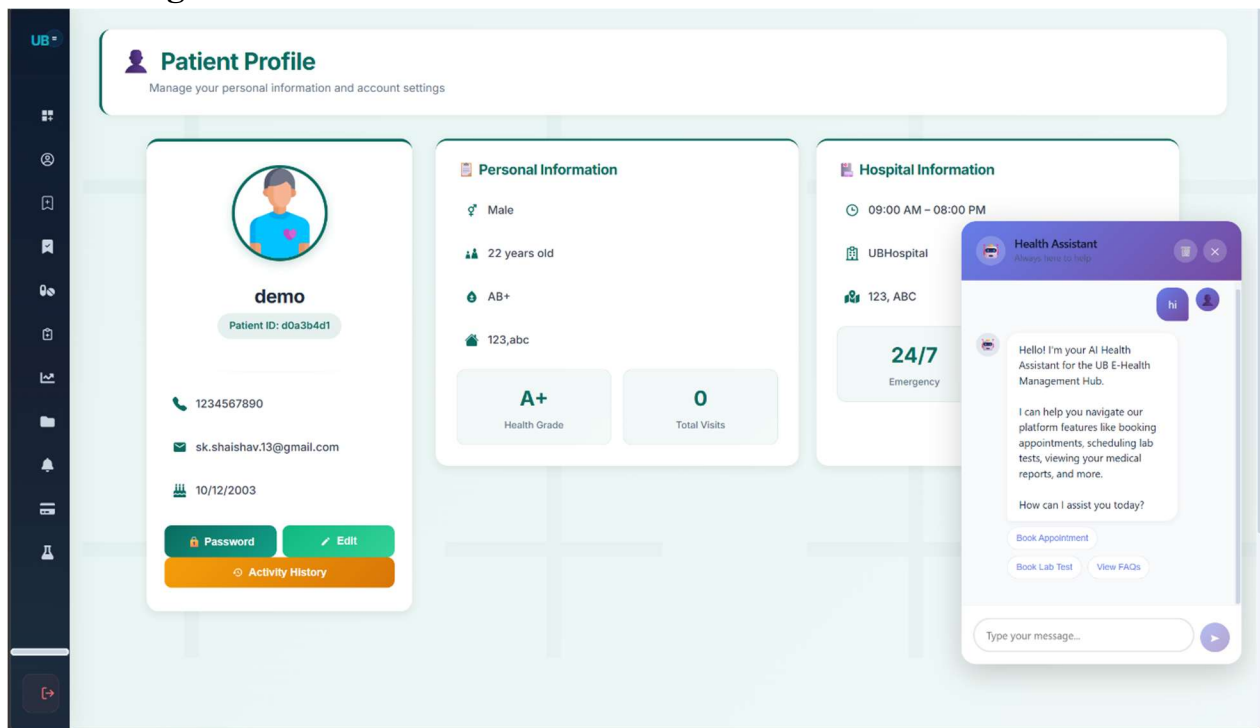
The Patient Dashboard is a user interface for a healthcare application. It features a dark blue sidebar on the left with various icons for navigation. The main content area has a light blue background with a grid of cards. At the top, a welcome message 'Welcome back, demo!' is displayed. Below this, there are four cards showing statistics: '0 Upcoming Appointments', '3 Active Medications', '0 Pending Lab Tests', and '0 Total Reports'. A 'Health Score' of 95 is shown in the top right. A 'Quick Actions' section follows, containing five cards: 'Book Appointment' (Schedule a visit with a doctor), 'My Medications' (View prescriptions & refills), 'Lab Tests' (Book lab tests & view results), 'My Reports' (Access medical reports), and 'Health Trends' (Track your health metrics). An 'Emergency' card with a 'COMING SOON' badge is also present, with the text 'Request ambulance service'.

Appointment Booking Form:

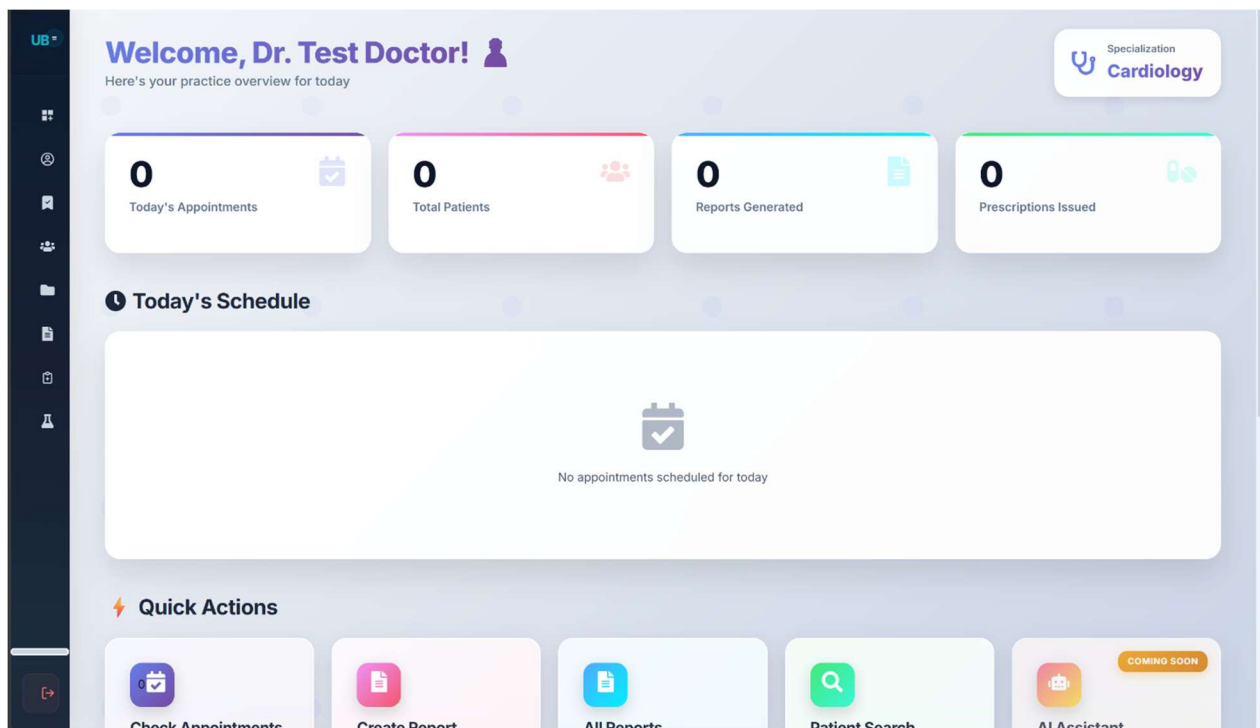


The Appointment Booking Form is a user interface for booking a doctor's appointment. It features a dark blue sidebar on the left with various icons for navigation. The main content area has a light blue background. At the top, there are two tabs: 'Doctor Appointment' (selected) and 'Lab Test'. Below the tabs, a 'Book Appointment' section is displayed, with the subtitle 'Select Department & Doctor'. The form contains several input fields: 'Department' (a dropdown menu with 'Select' as the placeholder), 'Doctor' (a dropdown menu with 'Select' as the placeholder), 'Fees' (a text input field), 'Problem' (a text input field with the placeholder 'Describe your issue'), and 'Date & Time' (a date input field with the placeholder 'dd-mm-yyyy' and a time dropdown menu with 'Select Time' as the placeholder). A 'Proceed to Payment' button is located at the bottom of the form.

Profile Page and AI-Chatbot:



Doctor Dashboard:



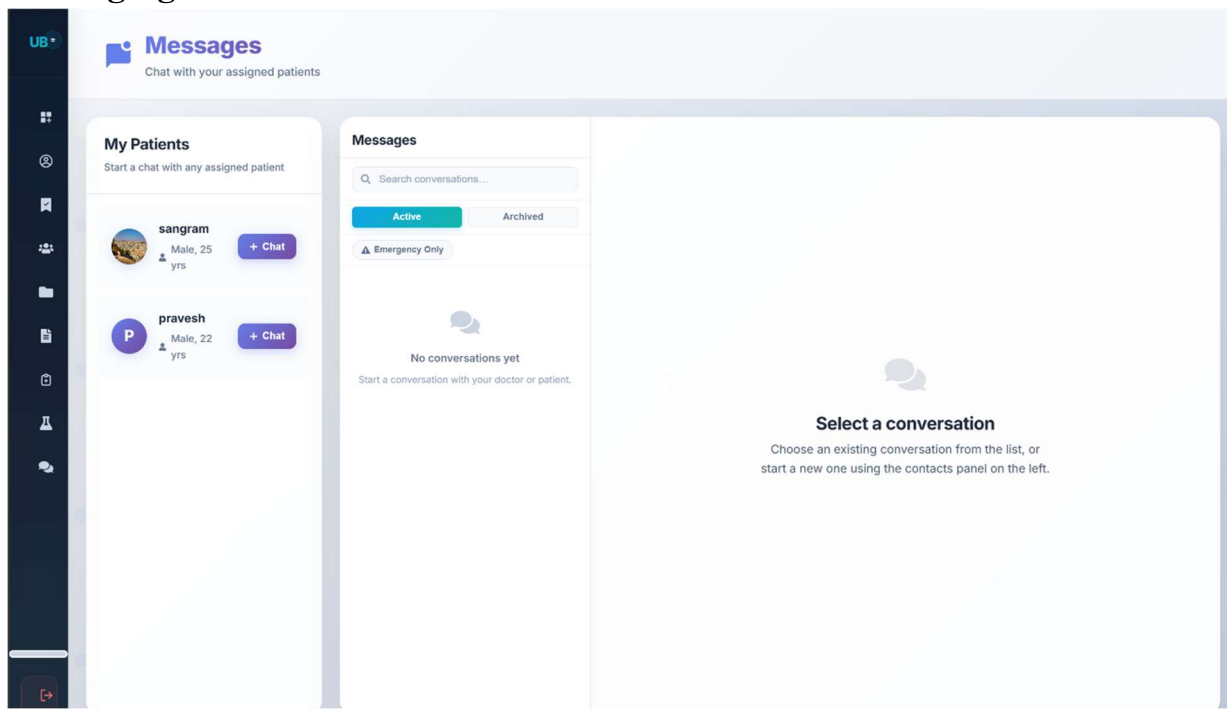
Patient Medication:

The screenshot shows the 'My Medications' app interface. At the top, there's a header with the app name and a subtitle 'Track and manage your prescribed medications'. Below this, three summary cards display '3 Total Medications', '3 Active', and '0 Completed'. A filter bar allows switching between 'All Medications', 'Active Only', and 'Completed', with a 'Clear Completed' button. The main list shows two medications: 'azithromizen' and 'paracetamol'. The 'azithromizen' entry is expanded, showing a diagnosis of 'cold and allergy' (dated 16/2/2026) and a doctor named 'Sangram'. Below the medication name, three tabs show 'DOSAGE' (1), 'FREQUENCY' (2), and 'DURATION' (3). At the bottom of the entry are buttons for 'View Full Prescription' and 'Mark as Completed'. The 'paracetamol' entry is partially visible below.

AI-Report Interpreter:

The screenshot displays the 'Your Report Simplified' AI-powered report. The report is divided into three main sections: 'Summary', 'Your Diagnosis Explained', and 'Your Vital Signs'. The 'Summary' section states that the user has been diagnosed with a cold and allergies, and that the doctor recommends rest and medication. The 'Your Diagnosis Explained' section provides a detailed explanation of 'Cold and Allergies', including what it means and why it matters. The 'Your Vital Signs' section shows three vital signs: Temperature (98.5°F), Blood Pressure (126/86 mmHg), and Glucose (82 mg/dL), each with a green checkmark indicating a normal reading. The background shows a blurred view of a medical record or dashboard.

Messaging:



AI Feature Outcomes

1. Patient Comprehension: Lab reports with complex parameters are translated into plain-language summaries that patients can understand without medical training.
2. Offline Availability: The Ollama fallback ensures AI features remain functional in environments without internet access, making the system viable for rural healthcare settings.
3. Structured Output: The JSON response format enables the frontend to display interpretations in organized sections: ExecutiveSummary, Parameter Explanations, Recommendations, and Questions for the Doctor.
4. Safety Disclaimer: Every AI interpretation includes a disclaimer reminding patients to their doctor, preventing over-reliance on AI medical advice.

Security Outcomes

- XSS Prevention: All user inputs sanitized. Script tags stripped.
- Injection Prevention: ObjectId validation prevents NoSQL injection.
- Unauthorized Access: Role middleware blocks cross-role access.
- Password Security: bcrypt hashing with salt rounds.
- File Upload Security: MIME type and size validation enforced.

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The UB E-Health Management Hub successfully demonstrates that a comprehensive, AI-powered healthcare management system can be built using open-source technologies on the MERN stack. The project addressed the core problem of fragmented, paper-based hospital workflows by delivering a unified digital platform that serves patients, doctors, laboratory personnel, and administrators through a single web application.

All planned objectives were achieved. The role-based authentication system correctly restricts access based on user type. The appointment and laboratory test booking modules streamline scheduling and reduce administrative overhead. The AI-powered report interpretation engine — with its three-tier fallback design using Google Gemini, Ollama, and a rule-based engine — successfully translates complex medical data into patient-friendly language, even in offline environments. The real-time doctor-patient chat system built on Socket.io and Redis enables asynchronous consultations that reduce unnecessary in-person visits. The integrated Razorpay payment gateway, automated email notification system, digital prescription management, and document sharing features collectively digitize the complete patient care lifecycle.

A key original contribution of this project is the dual-mode AI architecture that combines a cloud-based AI (Gemini) with a locally hosted model (Ollama), making intelligent medical report interpretation accessible in environments with limited or no internet connectivity. This design makes the system viable for deployment in rural clinics and community hospitals where cloud dependency is a barrier.

The system was tested end-to-end across all modules and performed within acceptable response time benchmarks. Security was implemented at every layer — JWT authentication, bcrypt password hashing, input validation, sanitization, and role-based access control — ensuring that sensitive patient data is protected.

In conclusion, this project establishes a working, production-ready prototype of an intelligent healthcare management platform that can meaningfully improve patient care quality, reduce

administrative burden, and make AI-assisted medical understanding accessible to non-technical users.

6.2 Future Scope

While the current system is fully functional, several enhancements are planned for future versions:

1. **Video Consultation (Telemedicine)** Integrate WebRTC-based video calling between doctors and patients directly within the platform, enabling full telemedicine capabilities without requiring third-party apps.
2. **Mobile Application** Develop a React Native mobile application for Android and iOS, giving patients and doctors access to all features on their smartphones with push notification support.
3. **Medical Image Analysis with AI** Extend the AI interpretation engine to analyze medical images such as X-rays, MRI scans, and CT scans using vision-capable AI models, providing preliminary observations to assist doctors.
4. **Wearable Device Integration** Connect with wearable health devices (smartwatches, glucose monitors, blood pressure cuffs) via APIs to automatically import real-time health metrics into the patient's health trends dashboard.
5. **Two-Factor Authentication (2FA)** Add an additional security layer using OTP-based two-factor authentication via SMS or authenticator apps, especially important for protecting sensitive medical records.
6. **Multi-Language Support** Add support for regional Indian languages (Hindi, Marathi, Tamil, Telugu, etc.) to make the platform accessible to patients who are not comfortable with English.
7. **Advanced Analytics Dashboard for Admins** Build a comprehensive analytics panel showing hospital-wide statistics — appointment trends, most booked specialties, revenue reports, lab test volumes, and AI usage metrics.
8. **Insurance Integration** Integrate with health insurance providers to allow patients to submit claims directly from the platform and track claim status in real time.
9. **Drug Interaction Checker** Add a medication safety module that checks for known drug interactions when a doctor creates a prescription, alerting them to potentially dangerous combinations.

10. Progressive Web App (PWA) Convert the frontend into a Progressive Web App so it can be installed on devices, work offline for basic features, and send native push notifications without a separate mobile app.
11. Electronic Health Record (EHR) Export Implement HL7 FHIR standard-compliant data export so patient records can be shared with other hospital systems and EHR platforms.
12. Audit Trail and Compliance Reporting Build a full audit logging system for HIPAA/DPDP Act compliance, recording every access to patient data with timestamps, user IDs, and actions performed.

REFERENCES

The following official documentation sources were referenced during the design and development of this project:

Frontend Technologies

1. React.js Official Documentation: <https://react.dev/>
2. Redux Official Documentation: <https://redux.js.org/>
3. React Router v6 Documentation: <https://reactrouter.com/en/main>
4. Axios HTTP Client Documentation: <https://axios-http.com/docs/intro>
5. Chart.js Documentation: <https://www.chartjs.org/docs/latest/>
6. jsPDF Documentation: <https://artskydj.github.io/jsPDF/docs/>
7. React-Toastify Documentation: <https://fkhadra.github.io/react-toastify/introduction>

Backend Technologies

1. Node.js Official Documentation: <https://nodejs.org/en/docs>
2. Express.js Official Documentation: <https://expressjs.com/>
3. MongoDB Official Documentation: <https://www.mongodb.com/docs/>
4. Mongoose ODM Documentation: <https://mongoosejs.com/docs/>
5. Socket.io Official Documentation: <https://socket.io/docs/v4/>
6. Redis Official Documentation: <https://redis.io/docs/>
7. ioredis (Node.js Redis Client) Documentation: <https://github.com/redis/ioredis>
8. JSON Web Tokens (JWT) — jwt.io: <https://jwt.io/introduction>
9. jsonwebtoken npm Package Documentation: <https://github.com/auth0/node-jwebtoken>
10. bcrypt npm Package Documentation: <https://github.com/kelektiv/node.bcrypt.js>
11. Multer File Upload Middleware Documentation: <https://github.com/expressjs/multer>
12. Nodemailer Official Documentation: <https://nodemailer.com/about/>
13. node-cron Documentation: <https://github.com/node-cron/node-cron>
14. Winston Logger Documentation: <https://github.com/winstonjs/winston>
15. validator.js Documentation: <https://github.com/validatorjs/validator.js>
16. DOMPurify Documentation <https://github.com/cure53/DOMPurify>

AI and Machine Learning

1. Google Gemini API Documentation: <https://ai.google.dev/gemini-api/docs>
2. Google Generative AI Node.js SDK: <https://github.com/google-gemini/generative-ai-js>
3. Ollama Official Documentation: <https://ollama.com/>
4. Ollama REST API Reference: <https://github.com/ollama/ollama/blob/main/docs/api.md>

Development Tools and Standards

1. npm (Node Package Manager) Documentation: <https://docs.npmjs.com/>
2. Git Official Documentation: <https://git-scm.com/doc>
3. MongoDB Atlas Documentation: <https://www.mongodb.com/docs/atlas/>
4. CORS (Cross-Origin Resource Sharing) — MDN Web Docs:
<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
5. REST API Design Best Practices — MDN Web Docs: https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Client-side_web_APIs/Introduction
6. WebSocket API — MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API